

Task Recovery And Checkpointing Knowledge (TRACK): A Conceptual Framework for Recoverable Agentic Planning

Blas J. Galván
Engineer, PhD
Independent Researcher
June 2026

Abstract

The rapid development of agentic Artificial Intelligence (AI) has popularized the agentic loop—a repetitive cycle of observation, reasoning, action, and evaluation—as the dominant paradigm for autonomous task execution. However, the loop metaphor, while useful for short interactions, proves insufficient for complex engineering, technical, and organizational tasks that demand auditability, recoverability, and traceability across long execution horizons. Recent literature has independently identified several of these limitations: the absence of native pause-and-resume mechanisms in ReAct-style agents, the fragility of checkpoint-less agentic pipelines under tool failures, the degradation of knowledge state in long-horizon tasks, and the lack of structured recovery paths when semantic failures produce plausible but unsupported outputs.

This paper proposes **Task Recovery And Checkpointing Knowledge (TRACK)**, a conceptual framework for agentic planning in which each task is defined by an explicit contract specifying inputs, outputs, executor, success condition, validation criteria, error policy, and artifacts to persist. TRACK mandates that every task generate a persistent, validated, and versioned output before the agent may advance. When a task fails—whether due to technical faults or semantic inconsistencies—the methodology diagnoses the failure class, inserts corrective recovery tasks into the original plan, updates active references to valid artifacts, and formally resumes execution at the correct plan point. TRACK thus aims to transform agentic planning from a fragile sequence of prompts into a robust, auditable, and recoverable workflow.

This work is presented as a position paper grounded in a structured review of 2025–2026 literature spanning durable execution, self-healing orchestration, and artifact-first agent design. The paper does not report an implemented system or empirical validation; rather, it defines a conceptual framework, situates it within the current research landscape, and identifies the components of TRACK that, to the best of our knowledge, have not yet been integrated by any existing agent framework.

Keywords: Agentic AI; task planning; checkpoints; recovery; replanning; persistent artifacts; durable execution; self-healing orchestration; workflow execution; AI agents; task traceability; plan governance; semantic failure; TRACK.

1 Introduction

1.1 The Agentic Loop as Dominant Paradigm

Agentic Artificial Intelligence systems are increasingly described through the concept of the **loop**. In its simplest and most influential form, the ReAct (Reasoning + Acting) paradigm interleaves intermediate reasoning with environment actions in a cycle of observation, deliberation, action, and evaluation [1]. This structure allows an AI agent to interact with its environment, invoke tools, evaluate intermediate results, and continue iterating until a termination condition is met. The popularity of the loop is understandable: it is concise, intuitive, and easy to communicate. Its success has spawned an ecosystem of loop-based frameworks, including Reflexion [2], Toolformer [3], and numerous multi-agent coordination systems built on iterative reasoning-and-acting patterns [4, 5].

Figure 1 summarizes the conceptual shift proposed in this paper: from loop-based repetition to recoverable agentic execution.

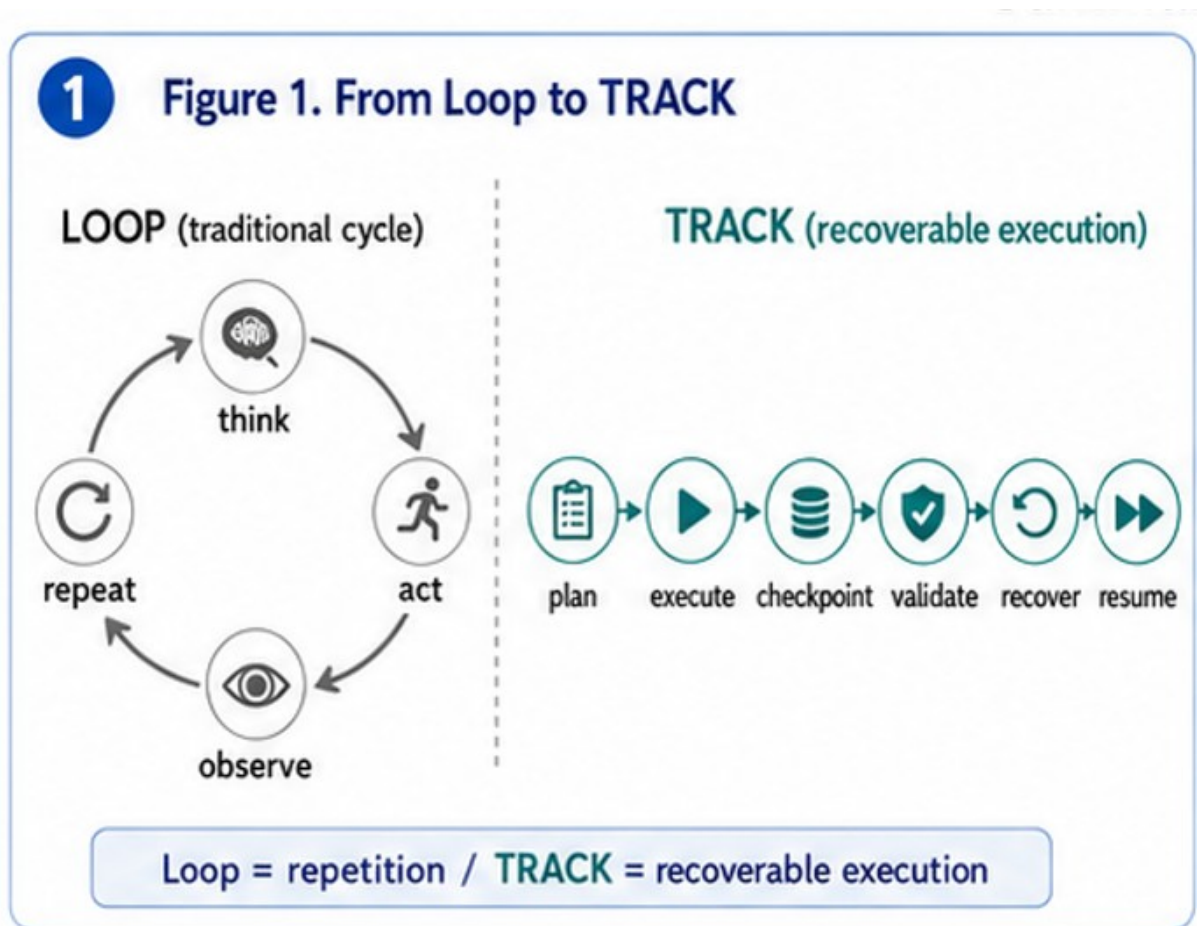


Figure 1: From the traditional agentic loop to TRACK. The loop emphasizes repeated cycles of thinking, acting, observing, and repeating, whereas TRACK structures agentic execution as a recoverable sequence of planning, execution, checkpointing, validation, recovery, and resumption.

However, as agentic systems transition from research demonstrations to production deployments, the limitations of the pure loop paradigm have become increasingly apparent. A growing

body of work across 2025–2026 has documented that complex, long-horizon tasks require more than repetition: they demand persistent state, explicit checkpoints, failure diagnosis, controlled recovery, and traceable execution—capabilities that simple loops do not natively provide.

1.2 Documented Limitations of Loop-Based Agents

Several independent lines of research have converged on the same diagnosis. Agents operating under ReAct-style loops lack native pause-and-resume mechanisms: there is no first-class facility to halt mid-execution for human approval, checkpoint state to durable storage, or resume from a known-good position after an upstream failure [6, 7].^{1 2} When failures occur—tool timeouts, malformed arguments, stale retrieval context, contradictory evidence, or wrong-but-plausible semantic outputs—loop-based agents default to uniform retry or improvisational correction, without structured failure diagnosis or targeted recovery [8].

The fragility of checkpoint-less agentic pipelines is particularly acute in production settings. Technical analyses have documented that long-running AI agent workflows—those orchestrating dozens of tool calls, spanning hours of execution, or requiring human review mid-run—cannot tolerate restarting from scratch upon failure [9].³ Checkpointing, combined with event-history replay and idempotent tool design, can transform these brittle pipelines into fault-tolerant workflows. Yet, as industry practitioners have argued, what current agent frameworks provide are *save points*—snapshots of state that the developer must manually detect the need for, trigger, and coordinate at scale—which falls short of production-grade durable execution [10].⁴

At the knowledge-state level, the degradation problem is equally acute. Annapureddy et al. propose StatePlane, addressing the challenge of maintaining coherent cognitive state across long interaction horizons when context windows are bounded [11]. Wu et al. introduce Crab, a semantics-aware checkpoint/restore runtime for agent sandboxes that optimizes checkpoint granularity by observing OS-visible effects at turn boundaries [12]. The ADEMA architecture introduces knowledge-state orchestration for long-horizon tasks where intermediate decisions remain implicit, the evolving knowledge state degrades, and interruption breaks the chain of evidence [13]. The InfiAgent framework tackles the same problem by externalizing persistent state into file-centric abstractions [14]. These works collectively demonstrate that conversational memory—the default state mechanism of loop-based agents—is structurally inadequate for sustained, revisable work.

Perhaps most critically, semantic failures—where an agent produces a syntactically valid but semantically wrong output—represent a class of errors that loop-based agents are poorly equipped to detect, let alone recover from. Babu and Agrawal demonstrate that in controlled semantic

¹Agents’ Codex, “AI Agent State Machines: Designing Persistent Workflows for Production,” April 2026. <https://agentscodex.com/posts/2026-04-10-ai-agent-state-machines-designing-persistent-workflows-for-production/>

²AgentMarketCap, “State Machine vs. ReAct: The Agent Architecture Choice That Defines Production Reliability,” April 2026. <https://agentmarketcap.ai/blog/2026/04/10/state-machine-vs-reactive-agent-architectures-langgraph-react>

³Zylos Research, “AI Agent Workflow Checkpointing and Resumability,” March 2026. <https://zylos.ai/research/2026-03-04-ai-agent-workflow-checkpointing-resumability>

⁴Diagrid, “Why Checkpoints Aren’t Durable Execution,” 2026. <https://www.diagrid.io/blog/checkpoints-are-not-durable-execution-why-langgraph-crewai-google-ai-and-others-fall-short-for-production-agent-workflows>

silent-failure settings, non-verifying agents return wrong-but-plausible outputs between 13.2% and 17.6% of the time, while verifier-guided recovery reduces silent failures to 0.0% [8]. This result underscores a fundamental gap: runtime monitoring of explicit failure signals (timeouts, exceptions) is insufficient; semantic validation must be part of the execution and recovery loop. Yang et al. further explore semantic recoverability for structured tool agents, distinguishing between mechanical rollback and context-aware recovery in commitment-sensitive settings [27].

1.3 The Emerging Landscape of Recoverable Agentic Execution

The response to these limitations has been rapid and multi-directional. At the infrastructure layer, durable execution engines—Temporal, Restate, Inngest, Hatchet, DBOS, Cloudflare Workflows, AWS Lambda Durable Functions, and Azure Durable Task—have emerged as reliability substrates for production AI agents, offering automatic state persistence, automatic retries, and crash recovery [15, 16].⁵ ⁶ At the orchestration layer, self-healing architectures map observable failure signals to inferred failure classes, select targeted recovery actions under explicit budgets, and verify recovered trajectories [8, 17, 18]. At the state layer, Rosen and Rosen propose artifact-first design, arguing that intermediate work products—evidence maps, claim structures, criteria, assumptions, plans—should be treated as first-class durable objects with identity, versioning, dependency awareness, and supersession semantics [19].

At the framework layer, LangGraph provides node-level checkpointing via PostgresSaver and RedisSaver [20],⁷ while Microsoft Agent Framework reached 1.0 GA in April 2026 with built-in workflow checkpoints [21].⁸ Google ADK 2.0 offers session-level state management, and CrewAI has crossed 2 billion agentic executions [22].⁹ Yet, as comprehensive comparisons document, no framework reviewed appears to implement the complete cycle of task contracts, mandatory validation, failure-class diagnosis, targeted recovery task insertion, active reference updating, and explicit resume-point governance [23, 24].¹⁰ ¹¹

Crucially, it must be noted that durable execution engines address the **infrastructure** layer—guaranteeing that code completes despite crashes—while the frameworks reviewed address the **orchestration** layer—coordinating model calls, tool invocations, and state transitions. Neither layer, however, currently provides a **planning methodology** that treats the plan itself as a recoverable governing structure. This distinction is essential: durable execution ensures that *what was started finishes*; a recoverable planning methodology ensures that *what was planned can be diagnosed, corrected, and resumed at the right point*.

⁵Zylos Research, “Durable Execution for AI Agent Runtimes,” April 2026.

<https://zylos.ai/research/2026-04-24-durable-execution-agent-runtimes/>

⁶Inngest, “Durable Execution: The Key to Harnessing AI Agents in Production,” 2025.

<https://www.inngest.com/blog/durable-execution-key-to-harnessing-ai-agents>

⁷LangChain, “LangGraph Persistence,” 2026.

<https://docs.langchain.com/oss/python/langgraph/persistence>

⁸Microsoft, “Agent Framework Workflows—Checkpoints,” 2026.

<https://learn.microsoft.com/en-us/agent-framework/user-guide/workflows/checkpoints>

⁹Efflow, “AI Agent Frameworks Compared 2026,” 2026.

<https://efflow.com/articles/ai-agent-frameworks-compared-2026>

¹⁰DeepResearch Ninja, “AI Agent Frameworks: A Comparative Analysis,” May 2026.

<https://depresearch.ninja/2026/05/AI-Agent-Frameworks-A-Comparative-Analysis-of-DSPy-Claude-Agent-SDK-OpenAI-Agents-SDK-CrewAI-AutoGen-LangGraph-and-Google-ADK/>

¹¹RaftLabs, “AI Agent Frameworks: LangGraph, crewAI, ADK Compared,” 2026.

<https://www.raftlabs.com/blog/ai-agent-framework-comparison>

1.4 Research Gap and Contribution

The preceding review reveals a landscape rich in components but lacking in integration. Durable execution provides infrastructure reliability but does not prescribe task-level planning methodology. Self-healing orchestration addresses failure recovery but does not define planning contracts. Artifact-first design provides state semantics but does not specify execution governance. Checkpointing frameworks save state but delegate recovery coordination to the developer. To the best of our knowledge, no existing system or methodology combines all of these elements into a unified planning framework that treats the plan itself as the central operating structure of the agent.

This paper introduces **Task Recovery And Checkpointing Knowledge (TRACK)** as a conceptual framework to fill this gap. TRACK is a methodology for designing agentic plans in which:

- (1) Every task is defined by an explicit contract (inputs, outputs, executor, success condition, validation, error policy, artifacts to persist);
- (2) Every task must produce persistent, versioned, and validated outputs before the agent advances;
- (3) Failures—both technical and semantic—trigger diagnosis, targeted recovery task insertion, active reference updates, and formal resumption of the original plan;
- (4) The plan structure itself serves as the central governance mechanism for execution, not merely as a prompt or suggestion.

The central thesis of TRACK is:

A robust agentic plan is not just a sequence of instructions. It is a recoverable execution structure with persistent checkpoints, explicit artifacts, validation rules, failure diagnosis, and controlled replanning. The agent must not depend on conversational memory to resume a plan; it must depend on persistent, versioned, and referable artifacts.

It is important to state clearly what TRACK is not. **TRACK is not a durable execution engine** (such as Temporal or Azure Durable Task); it is not an agent framework (such as LangGraph or CrewAI); and it is not an empirically validated system. **TRACK is a recoverable planning methodology**—a conceptual framework that defines how agentic plans should be structured, executed, checkpointed, and recovered.

1.5 Structure of This Paper

The remainder of this paper is organized as follows. Section 2 describes the research methodology employed for the literature review. Section 3 analyzes the principal problems and architectural proposals that motivate TRACK. Section 4 presents the complete TRACK framework, including its core principles, plan architecture, component definitions, recovery mechanisms, and a worked example. Section 5 discusses the relationship between TRACK and adjacent paradigms, presents a comparative capability analysis, identifies threats to validity, and outlines future work. Section 6 offers concluding remarks.

2 Research Methodology

This work follows a structured exploratory literature review designed to map the current state of recoverable agentic planning and identify gaps that TRACK addresses. The review was conducted in June 2026 and covers sources published or made available between January 2025 and June 2026, with select foundational references from earlier years included where they provide essential context.

2.1 Search Strategy

Search terms were organized into seven semantic clusters, each corresponding to a core capability dimension of TRACK (Table 1).

Table 1: Search clusters and representative terms.

Cluster	Focus	Example Terms
C1	Agentic recovery & checkpointing	“agentic task recovery checkpointing”, “recoverable agentic workflows”
C2	Loop limitations	“agentic loop limitations”, “ReAct loop limitations”
C3	State & knowledge management	“AI agent state management”, “knowledge state orchestration”
C4	Durable execution	“durable execution AI agents”, “async AI agent workflows checkpointing”
C5	Self-healing & failure recovery	“self-healing AI agents”, “LLM agent failure diagnosis”
C6	Artifacts & traceability	“artifact-based AI agent workflows”, “LLM agent observability traces”
C7	Frameworks	“LangGraph checkpointing persistence resume”, “LLM multi-agent orchestration survey 2026”

Searches were conducted using arXiv, Google Scholar, ACL Anthology, OpenReview, and general web search for technical documentation and industry articles. A total of 35+ distinct search queries were executed across the seven clusters.

2.2 Inclusion and Exclusion Criteria

Sources were included when they addressed at least one of the following TRACK-related capabilities: task-level checkpoints, durable or persistent execution state, agent failure recovery, self-healing orchestration, semantic failure handling, intermediate artifact persistence, execution traceability, or plan governance.

Sources were excluded when they: (a) focused exclusively on model-level checkpointing for training or inference (GPU/weight checkpointing) without relevance to agentic workflows; (b) addressed general AI agent construction without a specific focus on reliability, recovery, or state persistence; (c) were purely marketing materials with no technical depth; or (d) duplicated content already covered by a more authoritative or comprehensive source.

2.3 Source Classification

Each retained source was classified by type to ensure appropriate weighting in the analysis (Table 2).

Table 2: Source classification by type and analytical weight.

Type	Count	Examples
Academic paper (peer-reviewed)	3	Yao et al. (ICLR 2023), InfiAgent (ACL 2026)
Academic preprint (arXiv)	10	Babu & Agrawal (2026), Rosen & Rosen (2026)
Official documentation	3	LangGraph Persistence, MS Agent Framework
Technical article / industry analysis	8	Zylos Research, Diagrid, Inngest, Agents’ Codex
Comparative survey (third-party)	4	Effloow, DeepResearch Ninja, RaftLabs
Open-source repository	2	durable-agents, operational-evidence-plane

Academic preprints from 2026 should be interpreted with appropriate caution: they have not undergone formal peer review at the time of this writing.

2.4 Limitations of the Review

This review has several limitations that should be acknowledged. First, the search was not exhaustive: the field is moving rapidly, and relevant work may have been published after the search date or may not have been captured by the chosen search terms. Second, source quality varies significantly: arXiv preprints, official documentation, and technical blog posts carry different levels of authority. Third, the classification of sources by relevance to TRACK involves subjective judgment. Finally, the review was conducted by a single researcher. An independent replication with multiple reviewers and a formal systematic review protocol would strengthen the evidence base.

3 Problem Analysis and Related Proposals

This section examines the structural problems that loop-based agentic execution fails to address and surveys the principal proposals that have emerged to solve them. The analysis is organized around five capability gaps.

3.1 The State Persistence Gap

The most fundamental limitation of loop-based agents is their reliance on conversational memory as the primary—and often only—mechanism for preserving execution context. In a ReAct loop, the agent’s state is implicitly encoded in the transcript of model calls, tool outputs, and observations. When the transcript exceeds the context window, earlier state is truncated. When a session ends, all intermediate state disappears.

The consequences of this gap are documented across multiple studies. ADEMA demonstrates that in long-horizon knowledge synthesis tasks, the knowledge state degrades as decisions accumulate without explicit representation [13]. StatePlane formalizes this as a bounded-context problem, showing that existing approaches treat memory as static storage and fail to preserve decision-relevant state [11]. A recent survey on LLM agent memory notes that most approaches

treat memory as retrieval rather than as active, maintained state with known currency and authority [25].

The implication is structural: agentic systems need a state model that distinguishes between transient working memory and durable maintained state—artifacts that persist across steps, sessions, and recovery events, with known identity, version, status, and authority.

3.2 The Checkpointing Gap

Closely related is the absence of structured checkpointing. While an agent loop may occasionally save intermediate results to files, there is no systematic guarantee that every meaningful execution boundary produces a recoverable checkpoint [7]. The operational consequence is clear: a task orchestrating dozens of tool calls and spanning hours of execution cannot tolerate restarting from scratch on failure [9]. Crab demonstrates that checkpoint granularity can be optimized by observing that agents naturally alternate between local tool execution and waiting for the next LLM response [12].

However, checkpointing alone does not constitute durable execution [10]. A saved state snapshot requires the developer to detect failures, decide when to use the checkpoint, trigger restoration, and coordinate at scale. Production-grade durability demands that the runtime itself guarantee workflow completion, automatic recovery, and progress preservation.

3.3 The Failure Diagnosis Gap

Conventional software systems define failures primarily as exceptions or crashes. Agentic systems introduce a broader class of *semantic* failures: the answer is plausible but unsupported; the summary does not reflect the source; an LLM invents information; a tool succeeds technically but produces useless output [8, 26].

Existing recovery mechanisms are typically failure-agnostic. Retry mechanisms repeat failed calls without determining whether the failure is transient or structural. Full replanning discards useful partial state. Babu and Agrawal address this gap by formulating self-healing orchestration with explicit failure classes and targeted recovery actions under explicit budgets, achieving 98.8% task success compared with 94.5% for retry-only and 93.8% for full replanning [8]. The DART framework further distinguishes between mechanical rollback and semantic recoverability [27]. Zhu et al. introduce the ToolMaze benchmark for evaluating dynamic replanning and anomaly recovery [28].

3.4 The Intermediate Artifact Gap

Most agentic systems privilege final outputs while treating intermediate work as ephemeral. As Rosen and Rosen argue, a final answer is often a lossy projection over upstream state: a reconstructed rationale is not the same as preserved lineage [19]. They formalize a data model where intermediate artifacts are typed, structured, addressable, versioned, dependency-aware, authoritative, and consumable by downstream computation.

The Evidence Spine concept independently converges on the same need, defining the minimum

governed continuity required to answer audit questions about agentic runs.¹² Wang et al. provide a comprehensive survey on evidence tracing and execution provenance in LLM agents [29].

3.5 The Plan Governance Gap

The final gap concerns the status of the plan itself. In loop-based agents, the plan is typically a prompt-generated sequence of steps that the agent may follow, modify, or abandon. The plan is advisory, not governing. LangGraph addresses this partially by encoding plans as state graphs with explicit node transitions [20]. Microsoft Agent Framework encodes workflows with defined steps and checkpoint boundaries [21]. However, neither framework elevates the plan to the status of a governing contract that persists across recovery events and maintains its own state register.

3.6 Synthesis: Requirements for a Recoverable Planning Methodology

Taken together, the five gaps define a set of requirements that any methodology for recoverable agentic planning should satisfy:

- (1) **Task contracts.** Every unit of execution should specify its inputs, outputs, executor, success conditions, validation criteria, error policy, and artifacts to persist.
- (2) **Mandatory checkpoints.** No task should be considered complete until its output has been generated, stored, versioned, validated, and registered.
- (3) **Persistent artifacts.** Intermediate work products should be maintained as typed, addressable, versioned, dependency-aware, and authoritative objects.
- (4) **Failure diagnosis.** The system should distinguish among failure classes and avoid applying uniform recovery to heterogeneous failures.
- (5) **Targeted recovery.** Recovery actions should be selected based on the inferred failure class, execution state, and recovery budget.
- (6) **Active reference management.** After recovery, the system should explicitly update which artifact versions are active and which are superseded.
- (7) **Resume governance.** The system should return to the exact point in the original plan using updated active references.
- (8) **Plan as governing structure.** The plan itself should persist as the central operating structure, with state register, artifact register, and formal recovery semantics.

The five gaps identified in the analysis are summarized in Figure 2.

TRACK is designed to satisfy all eight requirements within a unified conceptual framework.

¹²Agent Axiom, “Evidence Spine,” 2026.

<https://github.com/agent-axiom/agent-arch/blob/main/docs/book/part-v/evidence-spine.en.md>

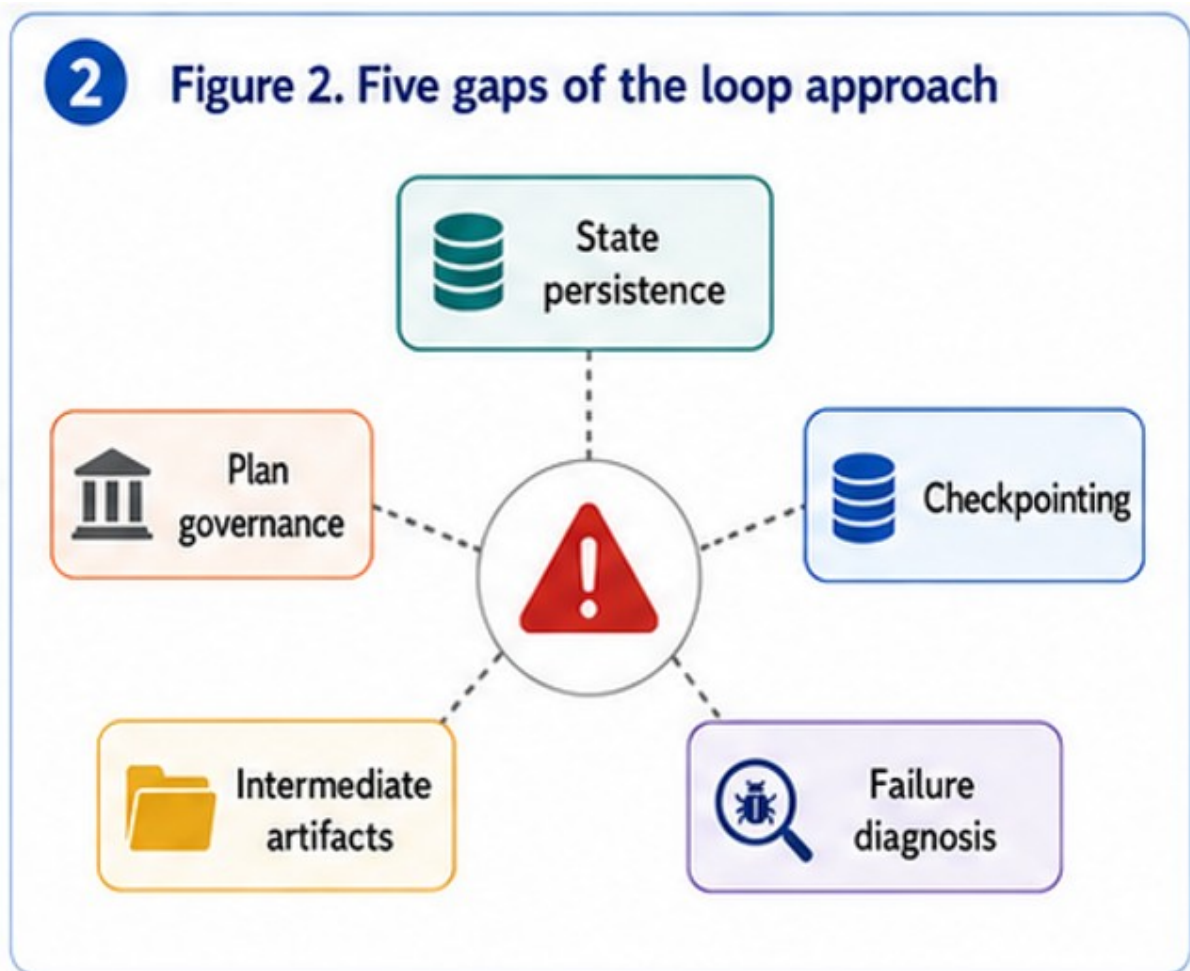


Figure 2: Five structural gaps of loop-based agentic execution. Loop-based agents may support repetition, but they do not natively provide persistent state, systematic checkpointing, semantic failure diagnosis, durable intermediate artifacts, or plan-level governance.

4 The TRACK Framework

4.1 Definition and Core Principles

TRACK stands for **T**ask **R**ecovery **A**nd **C**heckpointing **K**nowledge. It is a conceptual framework for designing agentic plans in which each task behaves like a function with a contract, but also generates persistent knowledge about its own execution.

Compact definition: TRACK is a conceptual framework for agentic planning based on task contracts, mandatory checkpoints, persistent artifacts, failure diagnosis, controlled replanning, and recovery of the original execution path.

Principle 1: Every Task Has a Contract

Each task in a TRACK plan must explicitly define: `task_id`, `name`, `description`, `inputs`, `outputs`, `executor`, `success_condition`, `validation`, `next`, `on_error`, and `persist`.

T1:

```
name = Evaluate_User_Request
description = Read the user prompt and decide whether
              the request is executable.
inputs = {user_prompt}
outputs = {user_prompt, verdict, verdict_reasons}
executor = {LLM_default}
persist = {artifact.user_prompt.v1, artifact.verdict.v1}
success_condition = {verdict in ["accept", "reject"]}
next = {IF verdict = "accept" THEN T2 ELSE T10}
on_error = {exit}
```

Principle 2: Every Task Must Produce Persistent Outputs

No task is considered complete until its outputs have been: generated, stored, identified, versioned, validated, and registered. The basic rule is: *The agent must not depend on its conversation memory to resume a plan. It must depend on persistent, versioned, and referable artifacts.*

Principle 3: Checkpoints Are Mandatory

A checkpoint in TRACK is the formal record of a task’s execution outcome, recording: `task_id`, `status`, `input_refs`, `output_refs`, `executor`, `confidence`, `validation_status`, `timestamp`, `error_message` (if failed), `next_task`, and `resume_point` (if applicable).

Principle 4: Failures Are Not Only Technical

TRACK recognizes both technical failures (timeouts, exceptions, malformed arguments, infrastructure crashes) and semantic failures (unsupported answers, unfaithful summaries, hallucinations, low-confidence classifications, loss of artifact reference). This dual taxonomy is informed by the failure classifications in Babu and Agrawal [8] and the ToolMaze perturbation taxonomy [28].

Principle 5: Recovery Tasks Are Inserted, Not Improvised

When a task fails, the original plan is not discarded. TRACK opens a recovery branch: a sequence of diagnostic and corrective tasks inserted between the failed task and the original subsequent task:

Original plan: T1 -> T2 -> T3 -> T4 -> T5

Failure in T3:

```
T1 -> T2 -> T3
                |
            R3.1 Diagnose failure
            R3.2 Repair input
            R3.3 Retry task
            R3.4 Validate recovered output
                |
            Resume T4
```

Principle 6: The Active Reference Must Be Updated

After recovery, multiple versions of the same artifact may exist (v1 = failed, v2 = corrected, v3 = validated). The plan must explicitly know which version is active, preventing the agent from accidentally consuming an obsolete result. The formal resumption instruction is: `RESUME T4 WITH artifact.X.active`.

4.2 TRACK Architecture

The layered architecture of TRACK is shown in Figure 3.

4.3 Components of a TRACK Plan

A complete TRACK plan includes: plan header (name, description, inputs, outputs), commands/tools (primitive actions with contracts), control flow (IF, FOR EACH, WHILE, ON ERROR, RESUME), main tasks (with full contracts), recovery tasks (with `failed_task`, `failure_type`, `update_references`, `resume_fields`), state register (`current_task`, `status`, `completed/failed/inserted tasks`, `active_artifacts`, `next_task`), and artifact register (`artifact_id`, `version`, `status`, `created_by`, `used_by`, `lineage`).

The contract-based structure of a TRACK task is illustrated in Figure 4.

The recovery mechanism is represented in Figure 5: recovery tasks are inserted into the plan rather than improvised outside it.

4.4 Worked Example

To illustrate TRACK, consider T3 (Read Documents) returning a partial read: 3 of 4 documents succeed, the fourth fails with “`unsupported_format`.” The checkpoint records `status: partial` and routes to T3B (Manage Partial Reading), which decides to continue with limitations. The

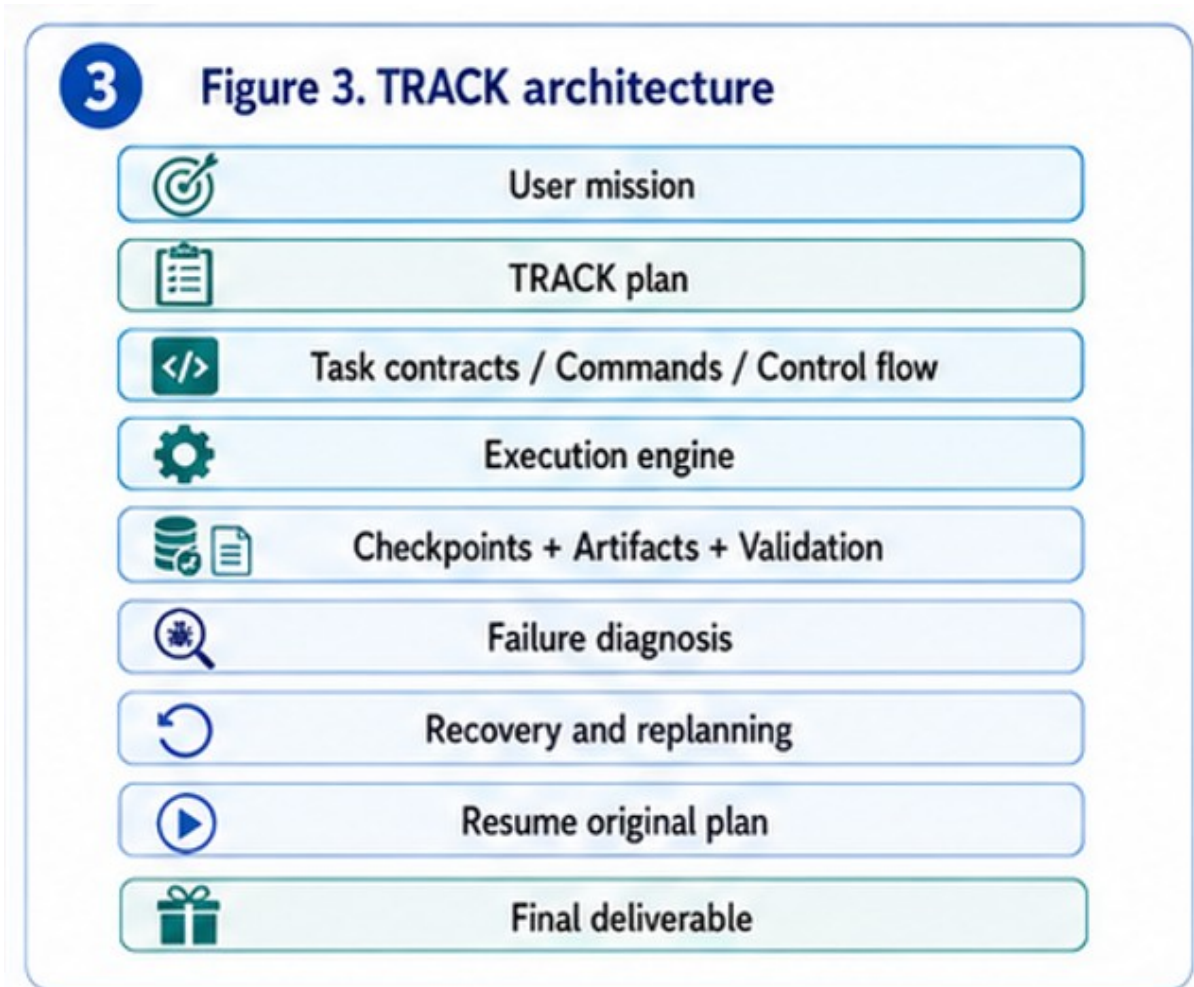


Figure 3: General architecture of the TRACK framework. The user mission is transformed into a TRACK plan, decomposed into task contracts, commands, and control flow, executed through an engine, checkpointed through persistent artifacts and validation, and recovered through failure diagnosis, replanning, and explicit resumption of the original plan.

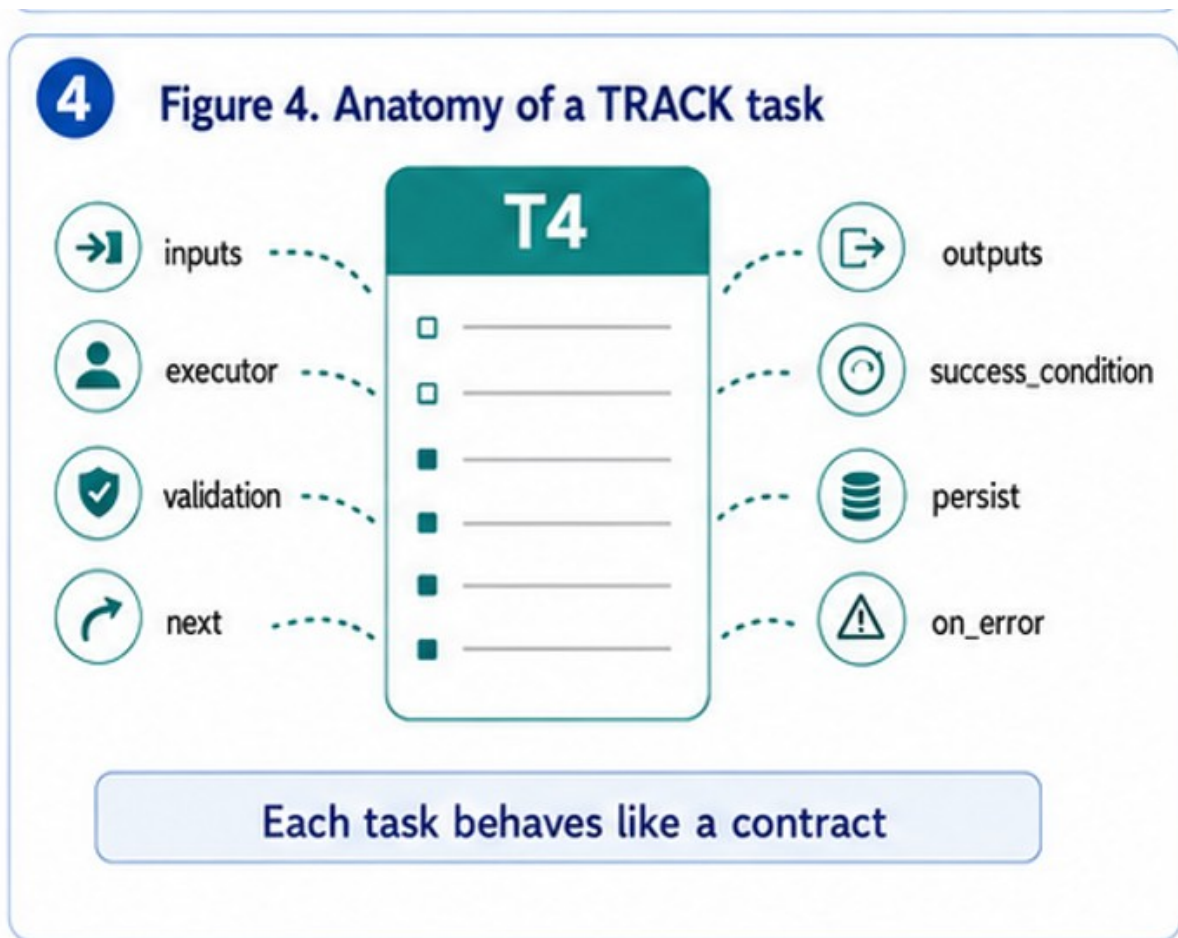


Figure 4: Anatomy of a TRACK task. Each task is treated as a contract that specifies inputs, outputs, executor, success condition, validation, persistence requirements, next transition, and error handling policy.

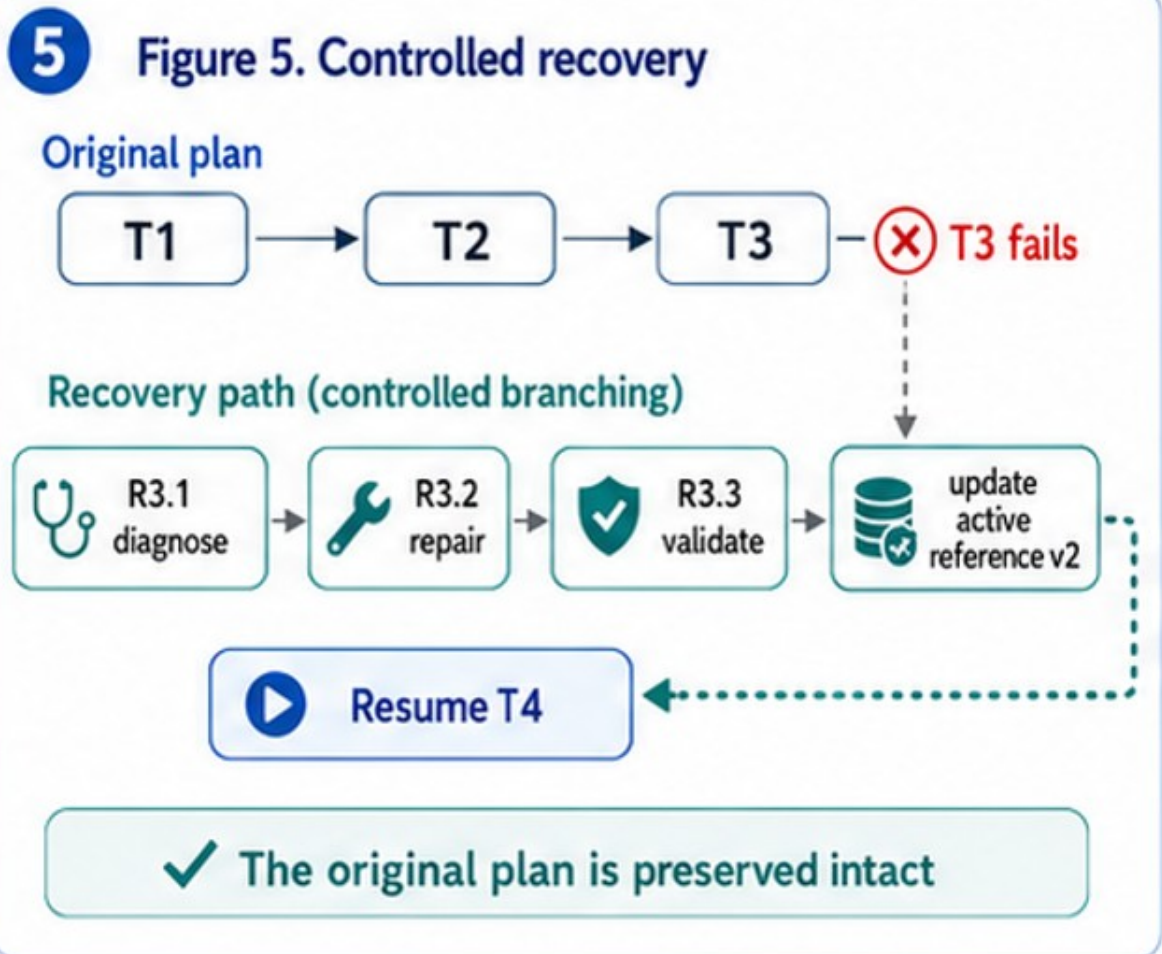


Figure 5: Controlled recovery in TRACK. When a task fails, the original plan is preserved while a recovery branch is inserted to diagnose the failure, repair the state, validate the recovered output, update the active artifact reference, and resume execution at the correct point.

plan proceeds through T4 (summaries), T5 (synthesis), T6 (validation), T7 (final file), and T8 (close)—all while carrying the limitations note. If the partial read had been more severe, T3B would route to R3 (Diagnose Read Failure) for format conversion or user clarification.

5 Discussion

5.1 TRACK Is Not a Durable Execution Engine

A critical distinction must be emphasized: **TRACK is not a durable execution engine**. Engines such as Temporal, Azure Durable Task, and Restate guarantee that code completes despite infrastructure failures. TRACK operates at a different layer: it is a **planning methodology** that defines how tasks should be structured and how recovery should be governed.

Figure 6 positions TRACK as a planning-governance layer that complements, rather than replaces, durable execution engines and agent orchestration frameworks.

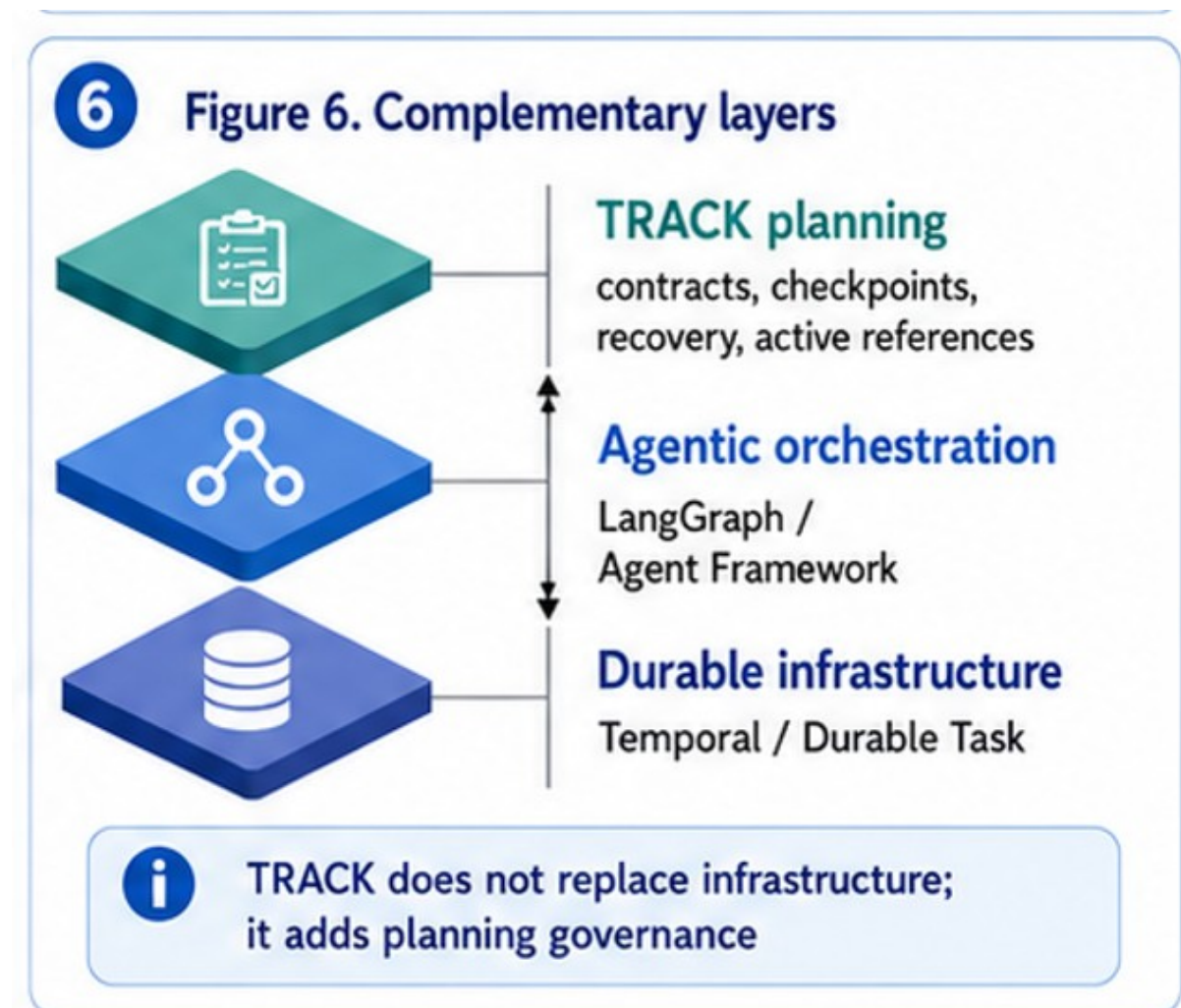


Figure 6: Complementary layers for recoverable agentic systems. Durable infrastructure provides execution reliability, agentic orchestration frameworks coordinate models and tools, and TRACK adds a planning-governance layer based on task contracts, checkpoints, recovery, and active references.

5.2 Comparative Capability Analysis

Table 3 presents a prudently calibrated comparison of TRACK’s capabilities against existing approaches, using four levels: **not addressed**, **partially addressed**, **strongly addressed**, and **unknown**.

Table 3: TRACK capabilities vs. existing systems.

Capability	ReAct	LangGraph	MS Agent	Temporal	Self-Healing	TRACK
Task-level contracts	–	Partial	Partial	–	–	Strong
Mandatory check-points	–	Strong	Strong	Strong	Partial	Strong
Persistent artifact versioning	–	Partial	Partial	Partial	–	Strong
Validation gates	–	–	–	–	Strong	Strong
Failure diagnosis (semantic)	–	–	–	–	Strong	Strong
Targeted recovery actions	–	–	–	–	Strong	Strong
Active reference management	–	–	–	–	–	Strong
Explicit resume points	–	Partial	Partial	Strong	–	Strong
Plan as governing artifact	–	Partial	Partial	–	–	Strong
Artifact lineage & supersession	–	–	–	–	–	Strong

Key: – = not addressed; Partial = partially addressed; Strong = strongly addressed.

5.3 Threats to Validity and Limitations

This work is presented as a conceptual framework and position paper. Several threats to validity must be acknowledged. **Construct validity:** TRACK’s operational semantics have not been validated through implementation or empirical study. **Internal validity (review):** The literature review is exploratory, not systematic. **External validity:** TRACK has been motivated primarily by examples from document processing and structured analytical tasks; applicability to other domains has not been assessed. **Implementation feasibility:** Practical deployment would require a formal grammar, an execution engine, durable storage, and integration with existing frameworks. **Framework classification:** The assessment in Table 3 reflects publicly documented features as of June 2026 and should be interpreted as a snapshot.

5.4 Future Work

The most important next step is a reference implementation—`track_engine.py`—that parses TRACK plans, executes tasks via LLM APIs, persists checkpoints and artifacts, and implements the recovery logic. A controlled experiment should compare a TRACK-guided agent against a loop-only agent on a complex, multi-step task with injected failures (e.g., FMECA generation, RAMS analysis). A formal grammar in YAML with JSON Schema validation should be defined. The failure taxonomy should be refined through empirical study of real agent failures. Human-in-the-loop integration should be formalized. Finally, a production-oriented implementation should integrate with Temporal or Azure Durable Task for infrastructure reliability and with LangGraph or Microsoft Agent Framework for model orchestration.

6 Conclusion

6.1 Summary

This paper has introduced **Task Recovery And Checkpointing Knowledge (TRACK)**, a conceptual framework for agentic AI planning that addresses five structural gaps: state persistence, checkpointing, failure diagnosis, intermediate artifacts, and plan governance. TRACK is grounded in six principles and provides a complete architecture designed to transform agentic planning from a fragile sequence of prompts into a robust, auditable, and recoverable workflow.

6.2 Positioning

TRACK emerges at a moment of convergence. Durable execution engines provide the infrastructure layer [16]. Self-healing orchestrators demonstrate that targeted recovery with verification outperforms uniform strategies [8]. Artifact-first design establishes durable intermediate state semantics [19]. Evidence tracing provides the vocabulary for auditability [29]. TRACK synthesizes these threads, adding task contracts, mandatory validation, recovery task insertion, and active reference management as a unified planning methodology.

It must be reiterated: **TRACK is not a durable execution engine; it is a recoverable planning methodology.** This distinction protects the novelty of TRACK while acknowledging its compatibility with the infrastructure and frameworks that have emerged in 2025–2026.

6.3 Final Formulation

The essential contribution can be expressed compactly:

$$\begin{aligned} \text{Loop} &= \text{think} \rightarrow \text{act} \rightarrow \text{observe} \rightarrow \text{repeat} \\ \text{TRACK} &= \text{plan} \rightarrow \text{execute} \rightarrow \text{checkpoint} \rightarrow \text{validate} \rightarrow \text{recover} \rightarrow \text{resume} \end{aligned}$$
$$\text{Loop} = \text{repetition.} \qquad \text{TRACK} = \text{recoverable execution.}$$

And its core rule:

No task is complete until its result has been saved, validated, and made referable by subsequent tasks.

This principle aims to transform agentic planning from an improvisational model into an engineering execution model—a transition that the production deployment of agentic AI increasingly demands.

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. “ReAct: Synergizing Reasoning and Acting in Language Models.” In *Proc. ICLR*, 2023.

- [2] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. “Reflexion: Language Agents with Verbal Reinforcement Learning.” In *NeurIPS*, vol. 36, pp. 8630–8652, 2023.
- [3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. “Toolformer: Language Models Can Teach Themselves to Use Tools.” In *NeurIPS*, vol. 36, pp. 68539–68551, 2023.
- [4] Q. Wu et al. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” arXiv:2308.08155, 2023.
- [5] S. Hong et al. “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework.” arXiv:2308.00352, 2023.
- [6] Agents’ Codex. “AI Agent State Machines: Designing Persistent Workflows for Production.” April 2026. [Technical article]
- [7] AgentMarketCap. “State Machine vs. ReAct: The Agent Architecture Choice That Defines Production Reliability.” April 2026. [Technical article]
- [8] R. S. Babu and A. Agrawal. “Self-Healing Agentic Orchestrators for Reliable Tool-Augmented LLM Systems.” arXiv:2606.01416, June 2026.
- [9] Zylos Research. “AI Agent Workflow Checkpointing and Resumability.” March 2026. [Technical article]
- [10] Diagrid. “Why Checkpoints Aren’t Durable Execution.” 2026. [Technical article]
- [11] S. Annapureddy, J. Mulcahy, and A. P. Thamamani. “StatePlane: A Cognitive State Plane for Long-Horizon AI Systems.” arXiv:2603.13644, March 2026.
- [12] T. Wu, C. Chang, L. Cao, W. Gao, and W. Wang. “Crab: A Semantics-Aware Checkpoint/Restore Runtime for Agent Sandboxes.” arXiv:2604.28138, April 2026.
- [13] ADEMA. “ADEMA: A Knowledge-State Orchestration Architecture for Long-Horizon Knowledge Synthesis with LLM Agents.” arXiv:2604.25849, April 2026.
- [14] InfiAgent. “InfiAgent: An Infinite-Horizon Framework for General-Purpose Autonomous Agents.” In *Findings of ACL 2026*, 2026.
- [15] Zylos Research. “Durable Execution for AI Agent Runtimes.” April 2026. [Technical article]
- [16] Inngest. “Durable Execution: The Key to Harnessing AI Agents in Production.” 2025. [Technical article]
- [17] C. Cruz. “VIGIL: A Reflective Runtime for Self-Healing Agents.” arXiv:2512.07094, December 2025.
- [18] Graph-Based Self-Healing Tool Routing. “Graph-Based Self-Healing Tool Routing.” arXiv:2603.01548, March 2026.

- [19] J. Rosen and S. Rosen. “Intermediate Artifacts as First-Class Citizens.” arXiv:2605.12087, May 2026.
- [20] LangChain. “LangGraph Persistence.” 2026. [Official documentation]
- [21] Microsoft. “Agent Framework Workflows—Checkpoints.” 2026. [Official documentation]
- [22] Effloow. “AI Agent Frameworks Compared 2026.” 2026. [Comparative article]
- [23] DeepResearch Ninja. “AI Agent Frameworks: A Comparative Analysis.” May 2026. [Comparative article]
- [24] RaftLabs. “AI Agent Frameworks: LangGraph, crewAI, ADK Compared.” 2026. [Comparative article]
- [25] LLM Agent Memory Survey. “LLM Agent Memory: A Survey from a Unified Representation–Management Perspective.” OpenReview, 2026.
- [26] C. Winston and R. Just. “A Taxonomy of Failures in Tool-Augmented LLMs.” In *Proc. IEEE/ACM AST*, pp. 125–135, 2025.
- [27] K. Yang, P. Li, Z. Wu, K. Xu, H. Huang, and X. Huang. “DART: Semantic Recoverability for Structured Tool Agents.” arXiv:2605.23311, May 2026.
- [28] D. Zhu, X. Ma, Y. Shen, X. Li, Y. Zhao, S. Wang, L. Yan, and D. Yin. “When Tools Fail: Benchmarking Dynamic Replanning and Anomaly Recovery in LLM Agents.” arXiv:2606.05806, June 2026.
- [29] Y. Wang et al. “From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents.” arXiv:2606.04990, June 2026.